

Matemática 4 – 2025

Entrega Análisis y Regresión Lineal

Integrantes:

- Dellarupe Franco - 21239/0
- Pierini Fermin - 21044/1
- Spadari Pedro - 21586/8

Dataset elegido:

Para el desarrollo del trabajo elegimos el dataset ***mls.csv***, que contiene información sobre los equipos de la liga de fútbol profesional de Estados Unidos (MLS). Cada fila representa un club, con datos como el nombre, ciudad, año de incorporación, ubicación geográfica y capacidad del estadio. Además, en este caso se pretende comprender qué variables podrían explicar el tamaño del estadio de cada equipo.

Obtenido del siguiente enlace: <https://github.com/gavinr/usa-soccer/blob/master/mls.csv>

Ipynb de Google Collaboratory:

Adjuntamos el link al Ipynb de Google Collaboratory donde desarrollamos los códigos y donde los mismos pueden ser ejecutados.

 TP-MATE 4.ipynb

Regresión Lineal Simple

A. **Variable respuesta:**

- **stadium_capacity** (capacidad del estadio):

La elegimos como variable respuesta porque es cuantitativa y continua, lo que va de acuerdo con el modelo lineal. Además, tiene sentido en la realidad ya que el tamaño del estadio puede estar condicionado por factores históricos, geográficos o de infraestructura, y queremos analizar si alguna de esas variables explica parte de su variabilidad.

Variables predictoras:

Se seleccionan tres variables numéricas del dataset como candidatas a predictoras:

- **joined** (año de incorporación a la liga): la antigüedad de un club en la liga podría estar asociada a mayores inversiones o a la propia evolución del estadio a través de los años y, por lo tanto, a estadios con más capacidad.

- **latitude** (latitud de la ciudad sede): la ubicación geográfica puede reflejar diferencias culturales o demográficas que influyen en el tamaño del estadio.

- **longitude** (longitud de la ciudad sede): similar a la latitud, nos permite explorar si existen patrones regionales que impacten en la infraestructura.

Elegimos estas variables basándonos en que la regresión lineal simple busca explicar la variación de una variable dependiente a partir de una independiente, evaluando si existe una relación lineal aproximada entre ambas. En este caso, vamos a analizar si la capacidad de los estadios (Y) puede ser explicada, al menos parcialmente, por cada una de las variables predictoras seleccionadas (X).

B.

Y	$\hat{y}_i = \hat{\beta}_1 x_1 + \hat{\beta}_0$	$\hat{\sigma}^2$	R^2	r	$IC(\beta_1)$	$IC(\beta_0)$	$ICM(Y)$	$IP(Y)$
$x_1(joined)$								
$x_2(latitude)$								
$x_3(longitude)$								

$x_1(joined)$:

- $\hat{y}_i = 282.97231 \cdot joined_i + (-543837.41683)$
- $\hat{\sigma}^2 = 3.53695e + 07$
- $R^2 = 0.19918$
- $r = 0.44630$
- $IC(\beta_1) = (63.32491399426846, 502.6197140601319)$
- $IC(\beta_0) = (-985078.041399807, -102596.79225680744)$
- $ICM(Y) = (22382.6177505754, 26830.982249424695)$
- $IP(Y) = (12223.077335214371, 36990.52266478572)$

$x_2(\text{latitude})$:

- $\hat{y}_i = (-75.64628) \cdot \text{joined}_i + 27504.39727$
- $\hat{\sigma}^2 = 4.39732e + 07$
- $R^2 = 0.00438$
- $r = -0.06618$
- $IC(\beta_1) = (-517.1864545696451, 365.8938852052387)$
- $IC(\beta_0) = (10410.541433277325, 44598.253116126405)$
- $ICM(Y) = (22126.810600286146, 27086.789399713845)$
- $IP(Y) = (10798.803400076993, 38414.796599923)$

$x_3(\text{longitude})$:

- $\hat{y}_i = (-9.84724) \cdot \text{joined}_i + 23680.48716$
- $\hat{\sigma}^2 = 4.41357e + 07$
- $R^2 = 0.00070$
- $r = -0.02648$
- $IC(\beta_1) = (-153.74774270876054, 134.05325547740972)$
- $IC(\beta_0) = (9917.893487575308, 37443.08083082392)$
- $ICM(Y) = (22122.23402196174, 27091.36597803831)$
- $IP(Y) = (10773.322090378537, 38440.27790962151)$

Código:

<pre>import pandas as pd import numpy as np from scipy.stats import t # Cargar dataset df = pd.read_csv("mls.csv") # Variable respuesta y = df["stadium_capacity"] # Variables predictoras predictoras = ["joined", "latitude", "longitude"] # Función para regresión simple calculada a mano def regresion_simple(x, y, alpha=0.05): n = len(x) # Promedios x_mean = np.mean(x) y_mean = np.mean(y) # --- Cálculos iniciales --- Sxx = np.sum(x**2) - (np.sum(x)**2 / n) Syy = np.sum(y**2) - (np.sum(y)**2 / n) Sxy = np.sum(x * y) - (np.sum(x) * np.sum(y) / n) # Estimadores de las betas beta1 = Sxy / Sxx beta0 = y_mean - beta1 * x_mean # Valores ajustados y residuales y_sombrero = beta0 + beta1 * x residuos = y - y_sombrero # Suma de cuadrados del error (SSE)</pre>	<pre># --- Intervalos ICM(Y) e IP(Y) en x promedio SE_y_sombrero_mean = sigma_est * np.sqrt(1/n) SE_pred_mean = sigma_est * np.sqrt(1 + 1/n) y_sombrero_mean = beta0 + beta1 * x_mean ICM_Y = (y_sombrero_mean - t_value * SE_y_sombrero_mean, y_sombrero_mean + t_value * SE_y_sombrero_mean) IP_Y = (y_sombrero_mean - t_value * SE_pred_mean, y_sombrero_mean + t_value * SE_pred_mean) return { "Variable": x.name, "β̂₀": beta0, "β̂₁": beta1, "σ̂²": sigma2_est, "R²": R2, "r": r, "IC(β₀)": IC_beta0, "IC(β₁)": IC_beta1, "ICM(Y)": ICM_Y, "IP(Y)": IP_Y } # Ejecutar y mostrar resultados resultados = [] for var in predictoras: resultados.append(regresion_simple(df[var], y))</pre>
--	--

<pre> SSE = np.sum(residuos**2) # Estimador de sigma² (MSE) sigma2_est = SSE / (n - 2) sigma_est = np.sqrt(sigma2_est) # Coeficiente de determinación y correlación R2 = 1 - (SSE / Syy) r = np.sign(beta1) * np.sqrt(R2) # --- Intervalos de confianza de las betas --- t_value = t.ppf(1 - alpha/2, df=n-2) SE_beta1 = sigma_est / np.sqrt(Sxx) SE_beta0 = sigma_est * np.sqrt(1/n + (x_mean**2 / Sxx)) IC_beta1 = (beta1 - t_value * SE_beta1, beta1 + t_value * SE_beta1) IC_beta0 = (beta0 - t_value * SE_beta0, beta0 + t_value * SE_beta0) </pre>	<pre> tabla_resumen = pd.DataFrame(resultados) pd.set_option('display.precision', 5) print("\n=== Tabla resumen de regresión lineal simple (cálculo manual) ===") print(tabla_resumen) </pre>
--	---

C. La variable que mejor responde a la variable respuesta es la variable *‘joined’* ya que tiene el mayor R^2 , el cual explica aproximadamente el 20% de la variabilidad en la capacidad del estadio.

Además, su correlación ‘ r ’ indica una relación lineal positiva moderada: a medida que aumenta *joined*, también tiende a aumentar la capacidad del estadio.

Por otra parte, su intervalo de confianza de β_1 (63.3, 502.6) no contiene 0, lo que sugiere que el coeficiente de pendiente es estadísticamente significativo.

Y por último, tiene el menor $\hat{\sigma}^2$, es decir, menor variabilidad de los errores respecto al modelo.

Entonces, podemos decir que, la variable *joined* es la mejor predictora de *stadium_capacity* ya que muestra una relación positiva y significativa entre ambas variables → a medida que aumenta el valor de *joined*, también aumenta la capacidad estimada del estadio.

Regresión Lineal Múltiple

D. Para estimar los parámetros del modelo de regresión lineal múltiple se aplicó el algoritmo de descenso del gradiente, su objetivo es minimizar la función de costo (error cuadrático medio) de manera iterativa, actualizando los coeficientes en la dirección opuesta al gradiente.

La variable respuesta considerada fue la capacidad del estadio (*stadium_capacity*), y las variables predictoras fueron *joined* (año de incorporación a la liga), *latitude* y *longitude*.

El modelo estimado tiene la forma:

$$\hat{Y} = \beta_0 + \beta_1 \cdot \text{joined} + \beta_2 \cdot \text{latitude} + \beta_3 \cdot \text{longitude}$$

Luego de ejecutar el algoritmo con una tasa de aprendizaje adecuada y un número suficiente de iteraciones, los parámetros convergieron a los siguientes valores aproximados:

$$\beta_0 \approx -559559.8048$$

$$\beta_1 \approx 289.4041$$

$$\beta_2 \approx 45.9398$$

$$\beta_3 \approx -11.0811$$

Por lo tanto, la ecuación de regresión estimada es:

$$\hat{Y} = (-559559.8048) + 289.4041 \cdot \text{joined} + 45.9398 \cdot \text{latitude} + (-11.0811) \cdot \text{longitude}$$

Además, el valor de R^2 calculado es: $R^2 = 0.2018$

Estos resultados muestran que, según el modelo, la capacidad de los estadios tiende a aumentar con el año de incorporación y con la latitud, mientras que disminuye levemente con la longitud. El intercepto β_0 representa el valor base de la capacidad cuando las demás variables son cero, lo cual no tiene interpretación práctica directa, pero es necesario para ajustar el modelo.

Código usado para el cálculo:

```
import pandas as pd
import numpy as np

# Cargar datos
datos =
pd.read_csv("mls.csv") [ ["stadium_capacity", "joined",
"latitude", "longitude"] ].dropna().astype(float)

# Separar variables predictoras (X) y respuesta (y)
valores_X =
datos[["joined", "latitude", "longitude"]].values
valores_y =
datos["stadium_capacity"].values.reshape(-1,1)
num_obs, num_vars = valores_X.shape

# Calcular medias y desvíos estándar
medias_vars = np.zeros(num_vars)
```

```
# gradiente
grad = np.zeros((p,1))
for j in range(p):
    grad[j,0] =
sum(X[i,j]*(pred[i,0]-y[i,0]) for i in range(m)) / m
# actualización
for j in range(p):
    coef[j,0] -= alpha*grad[j,0]
# condición de corte
costo_act = calcular_costo(X,y,coef)
if abs(costo_prev-costo_act) < tol:
    return coef,costo_act,it+1
costo_prev = costo_act
return coef,costo_prev,max_iter

coef_norm,costo_final,iteraciones =
descenso_gradiente(matriz_diseño,valores_y)
```

```

desvios_vars = np.zeros(num_vars)
for j in range(num_vars):
    suma = sum(valores_X[i,j] for i in
range(num_obs))
    medias_vars[j] = suma / num_obs
    suma_cuad =
sum((valores_X[i,j]-medias_vars[j])**2 for i in
range(num_obs))
    desvios_vars[j] = (suma_cuad / num_obs)**0.5

# Normalizar y armar matriz de diseño
X_normalizado = np.zeros_like(valores_X)
for i in range(num_obs):
    for j in range(num_vars):
        X_normalizado[i,j] =
(valores_X[i,j]-medias_vars[j]) / desvios_vars[j]

matriz_diseño = np.zeros((num_obs,num_vars+1))
for i in range(num_obs):
    matriz_diseño[i,0] = 1
    for j in range(num_vars):
        matriz_diseño[i,j+1] = X_normalizado[i,j]

# Función de costo
def calcular_costo(X,y,coef):
    m = len(y)
    pred = np.zeros((m,1))
    for i in range(m):
        pred[i,0] = sum(X[i,j]*coef[j,0] for j in
range(X.shape[1]))
    return sum((pred[i,0]-y[i,0])**2 for i in
range(m)) / (2*m)

# Descenso de gradiente
def
descenso_gradiente(X,y,alpha=0.5,max_iter=10000,to
l=1e-7):
    m,p = X.shape
    coef = np.zeros((p,1))
    costo_prev = calcular_costo(X,y,coef)
    for it in range(max_iter):
        # predicciones
        pred = np.zeros((m,1))
        for i in range(m):
            pred[i,0] = sum(X[i,j]*coef[j,0] for j
in range(p))

```

```

# Reescalar a escala original
coef_originales = np.zeros_like(coef_norm)
for j in range(1,num_vars+1):
    coef_originales[j,0] = coef_norm[j,0] /
desvios_vars[j-1]
coef_originales[0,0] = coef_norm[0,0] -
sum(coef_norm[j,0]*(medias_vars[j-1]/desvios_vars[j-
1])) for j in range(1,num_vars+1))

# Calcular R² en escala original
matriz_original = np.zeros((num_obs,num_vars+1))
for i in range(num_obs):
    matriz_original[i,0] = 1
    for j in range(num_vars):
        matriz_original[i,j+1] = valores_X[i,j]

pred_final = np.zeros((num_obs,1))
for i in range(num_obs):
    pred_final[i,0] =
sum(matriz_original[i,j]*coef_originales[j,0] for j
in range(num_vars+1))

media_y = sum(valores_y[i,0] for i in
range(num_obs)) / num_obs
ss_res = sum((valores_y[i,0]-pred_final[i,0])**2 for
i in range(num_obs))
ss_tot = sum((valores_y[i,0]-media_y)**2 for i in
range(num_obs))
R2 = 1 - ss_res/ss_tot

# Mostrar resultados
nombres_coef = ["β0 (intercepto)","β1 (joined)","β2
(latitude)","β3 (longitute)"]
print("Coeficientes (escala original):")
for nombre,val in
zip(nombres_coef,coef_originales.flatten()):
    print(f" {nombre}: {val:.4f}")
print(f"\nIteraciones GD: {iteraciones}, costo final
J = {costo_final:.6f}")
print(f"R^2 (usando coeficientes GD): {R2:.4f}")

```

- E. Para este inciso se aplicó el método de mínimos cuadrados, que consiste en minimizar la suma de los errores al cuadrado entre los valores observados y los valores estimados por el modelo. En notación matricial, la solución se obtiene mediante la expresión: $\hat{\beta} = (X^T X)^{-1} X^T y$

El modelo ajustado a nuestro caso tiene la forma:

$$\hat{Y} = \beta_0 + \beta_1 \cdot \text{joined} + \beta_2 \cdot \text{latitude} + \beta_3 \cdot \text{longitude}$$

Los coeficientes estimados por el metodo de minimos cuadrados fueron:

$$\beta_0 \approx -559559.8423$$

$$\beta_1 \approx 289.4041$$

$$\beta_2 \approx 45.9398$$

$$\beta_3 \approx -11.0811$$

Por lo tanto, la ecuación de regresión estimada es:

$$\hat{Y} = (-559559.8423) + 289.4041 \cdot \text{joined} + 45.9398 \cdot \text{latitude} + (-11.0811) \cdot \text{longitude}$$

El coeficiente de determinación obtenido fue: $R^2 \approx 0.2018$

Comparación con descenso de gradiente:

Al comparar con el descenso de gradiente, se observó coherencia en los

coeficientes y en el R^2 , lo cual era esperable ya que ambos métodos minimizan el mismo criterio (error cuadrático medio). La diferencia principal es metodológica: el método de mínimos cuadrados obtiene la solución de forma cerrada, mientras que el descenso de gradiente lo hace iterativamente. En este conjunto de datos, ambos convergen al mismo resultado, validando la consistencia del ajuste.

Código utilizado para el cálculo:

```
import pandas as pd
import numpy as np

# Cargar datos
datos = pd.read_csv("mls.csv")
datos = datos[["stadium_capacity", "joined",
"latitude", "longitude"]].dropna().astype(float)

# Definir variable respuesta (y) y variables
predictoras (X)
valores_predictores = datos[["joined", "latitude",
"longitude"]].values
valores_respuesta =
datos["stadium_capacity"].values.reshape(-1, 1)

# Cantidad de observaciones y variables
num_observaciones = valores_predictores.shape[0]
num_variables = valores_predictores.shape[1]

# Construir matriz de diseño con columna de 1s para
el intercepto
matriz_diseño = np.zeros((num_observaciones,
num_variables + 1))
for i in range(num_observaciones):
    matriz_diseño[i, 0] = 1.0
    for j in range(num_variables):
        matriz_diseño[i, j + 1] =
valores_predictores[i, j]
```

```
# Calcular coeficientes beta = (XtX)^(-1) * Xty
coeficientes_estimados = np.zeros((num_variables
+ 1, 1))
for i in range(num_variables + 1):
    suma = 0.0
    for j in range(num_variables + 1):
        suma += XtX_inversa[i, j] * Xty[j, 0]
    coeficientes_estimados[i, 0] = suma

# Calcular valores ajustados y residuos
valores_estimados = np.zeros((num_observaciones,
1))
for i in range(num_observaciones):
    suma = 0.0
    for j in range(num_variables + 1):
        suma += matriz_diseño[i, j] *
coeficientes_estimados[j, 0]
    valores_estimados[i, 0] = suma

# Calcular R²
# Media de y
media_respuesta = 0.0
for i in range(num_observaciones):
    media_respuesta += valores_respuesta[i, 0]
media_respuesta /= num_observaciones

# Suma de cuadrados residual y total
suma_residuos = 0.0
```

```

# Calcular  $X^T * X$  y  $X^T * y$ 
Xt = matriz_diseño.T

# Producto  $XtX$ 
XtX = np.zeros((num_variables + 1, num_variables + 1))
for i in range(num_variables + 1):
    for j in range(num_variables + 1):
        suma = 0.0
        for k in range(num_observaciones):
            suma += Xt[i, k] * matriz_diseño[k, j]
        XtX[i, j] = suma

# Producto  $Xty$ 
Xty = np.zeros((num_variables + 1, 1))
for i in range(num_variables + 1):
    suma = 0.0
    for k in range(num_observaciones):
        suma += Xt[i, k] * valores_respuesta[k, 0]
    Xty[i, 0] = suma

# Invertir  $XtX$  (usamos numpy porque hacerlo a mano sería muy largo)
XtX_inversa = np.linalg.inv(XtX)

```

```

suma_total = 0.0
for i in range(num_observaciones):
    error = valores_respuesta[i, 0] - valores_estimados[i, 0]
    suma_residuos += error * error
    desviacion = valores_respuesta[i, 0] - media_respuesta
    suma_total += desviacion * desviacion

R2 = 1.0 - (suma_residuos / suma_total)

# Mostrar resultados
nombres_coeficientes = ["β0 (intercepto)", "β1 (joined)", "β2 (latitude)", "β3 (longitude)"]
print("Coeficientes estimados por Mínimos Cuadrados:")
for nombre, valor in zip(nombres_coeficientes, coeficientes_estimados.flatten()):
    print(f" {nombre}: {valor:.4f}")

print(f"\nCoeficiente de determinación R²: {R2:.4f}")

```

- F. La incorporación de las variables *latitude* y *longitude* al modelo múltiple no mejoró la estimación respecto al modelo simple con *joined*. En ambos casos, el coeficiente de determinación R^2 se mantuvo cercano a 0.20, lo que indica que la capacidad explicativa del modelo no aumentó de manera significativa. Esto significa que la variable *joined* sigue siendo la principal predictora de la capacidad del estadio, mientras que las otras variables no aportan información adicional relevante.